

This comprehensive blueprint bridges your current codebase with the enterprise capabilities of **Google's Agent Development Kit (ADK 2.0)**, **Agents CLI (agents-cli)**, and **MongoDB Atlas Model Context Protocol (MCP)**.

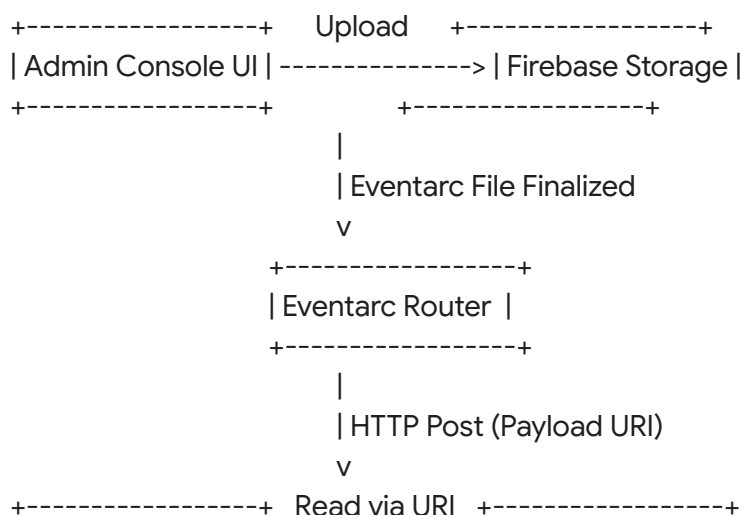
Part 1: Current Implementation Artifacts & Takeaways

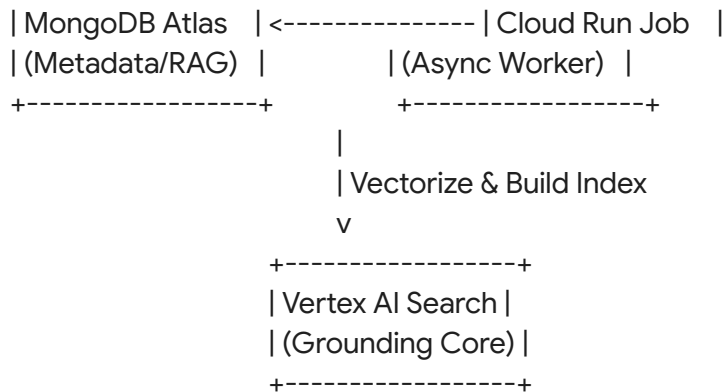
Based on our current codebase inspection, your infrastructure is strategically situated for this transition but requires a few direct adjustments:

- **Orchestrator Pattern Alignment:** Your `agents/orchestrator_agent/agent.py` currently relies on explicit custom routing hooks. This maps cleanly to the **Future Companion (Orchestrator Node)** pattern, but should be updated to use ADK 2.0's dynamic graph structures (Workflow and Node) rather than sequential linear loops.
- **MongoDB Engine Infrastructure:** Your `agents/mongodb_engine.py` maps low-level driver parameters manually. Moving to the **MongoDB Atlas MCP Server** means your agents can treat data collections as standard tools via standard function calling signatures, drastically decoupling database architecture from core model instructions.
- **Guardrails vs. Callbacks:** Your custom code in `agents/guardrails.py` uses linear pre-inference checks. To prevent structural breaks and control context footprints, these must be migrated directly to ADK's native `before_model_callback` hooks.

Part 2: Multimodal Ingestion, Analysis, & Library Schema

To support accurate data stitching between **Vertex AI Search (Grounding)** and **MongoDB Atlas (Structure)** without downloading files locally inside extraction runtimes, you can implement an asynchronous pipeline utilizing **Firebase Storage** as a shared transient lake and **Cloud Run Jobs** for serverless scaling.





Pipeline Flow:

1. **Ingestion:** The file is posted directly from the administration dashboard to a designated **Firestore Storage Bucket**.
2. **Triggering:** An **Eventarc Event** captures the `OBJECT_FINALIZE` state and passes an authenticated payload containing the asset's temporary public content URI to a target execution router.
3. **Analysis Worker:** A programmatic **Cloud Run Job** is kicked off asynchronously. Instead of downloading bytes, it transmits the public document stream link directly to the **Gemini 1.5 Pro API** via native file URI access patterns, asking the model to build complete content summaries, chapters, formulas, and structural layouts via structured JSON schemas (`response_mime_type="application/json"`).
4. **Stitching & Storage:** The output payload is split:
 - **Text content chunks** are ingested straight into **Vertex AI Search Data Store** to handle semantic grounding lookups during conversational turns.
 - **Structural metadata schemas** are committed directly into your unified **MongoDB collections** using **Gemini text-embedding models** for cross-entity correlations.

MongoDB Collection Architecture

To align with the comprehensive list of national education subjects (such as *Algebra and Statistics*, *Biology*, *Integrated Science*, and *Vocational Skills*), you can decouple items into a balanced Normalized/Embedded relational model inside Atlas:

```
// Collection: subjects
{
  "_id": "subj_algebra_stats",
```

```

    "name": "Algebra and Statistics",
    "emoji": "📊",
    "grade_levels": ["Grade 10", "Grade 11", "Grade 12"]
  }

  // Collection: books
  {
    "_id": "book_moe_alg_g10_t1",
    "subject_id": "subj_algebra_stats",
    "title": "High School Algebra and Analytic Geometry",
    "grade": "Grade 10",
    "term": "Term 1",
    "year": "2026",
    "language": "ar",
    "book_type": "core", // core (Student Book) vs supporting (Question Book)
    "source_url": "https://ellibrary.moe.gov.eg/content/...",
    "storage_path": "gs://fahem-academic-lake/moe/alg_g10_t1.pdf",
    "chapters": [
      {
        "id": "ch_1",
        "title": "Linear Equations and Matrices",
        "page_start": 4,
        "page_end": 28,
        "concepts": ["Matrix Inversion", "Determinants", "Cramer's Rule"]
      }
    ],
    "mindmap": {
      "root": "Matrices",
      "children": [{ "name": "Operations" }, { "name": "Transformations" }]
    },
    "keywords": ["matrix", "determinant", "linear system", "vector space"]
  }

  // Collection: question_bank
  {
    "_id": "q_mat_09832",
    "book_id": "book_moe_alg_g10_t1",
    "chapter_id": "ch_1",
    "page_reference": 14,
    "type": "MCQ", // MCQ vs TEXT_INPUT
    "complexity_rating": "intermediate",
    "question_text": "Given a matrix A where  $\det(A) = 0$ , what can be inferred about its inverse?",
    "distractors": [

```

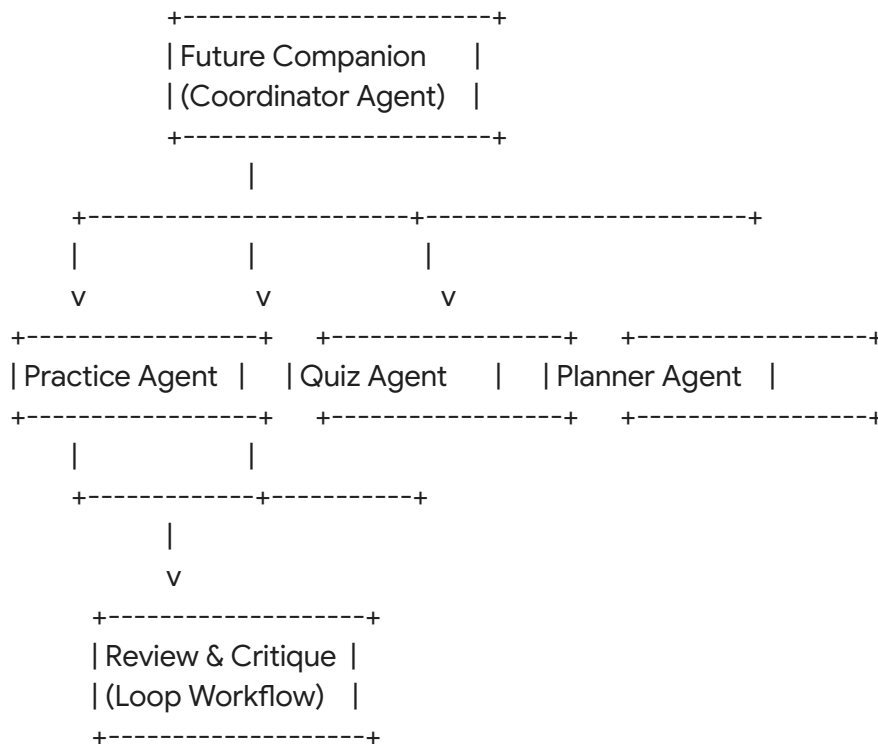
```

    "The inverse is equal to its transpose.",
    "The inverse is an identity matrix.",
    "The inverse can be found using Cramer's rule."
  ],
  "correct_answer": "The matrix does not possess an inverse.",
  "pedagogical_intent": "Testing understanding of singular matrices and inverse criteria.",
  "embedding": [0.0123, -0.0456, 0.2389, "Arabized Math vector elements..."]
}

```

Part 3: State-of-the-Art Agent Architecture

To manage this ecosystem without single-agent bottlenecks, your system requires a clear hierarchy utilizing **LLM Agents**, **Workflow Agents** (deterministic sequence controllers), and micro-scoped **Agents-as-Tools**.



1. Future Companion (Coordinator Pattern)

The entry hub of the academic environment. It maintains the absolute root level context, tracks

whether items are currently "magnetized" into context memory, evaluates performance parameters using structural analytical queries through the MongoDB MCP Toolset, and transfers conversational streams to sub-specialist nodes when explicitly invoked.

2. Practice Agent (Multi-turn ReAct Sub-Tree)

This sub-agent manages the core learning loops:

- **MCQ Sub-agent:** Dynamically queries the question_bank matching current chapter coordinates, formats outputs for clean interface rendering, and stores progress logs.
- **Text Practice Sub-agent:** Administers long-form open-ended questions. It includes specific metadata properties that disable copy-paste actions on the DOM level, forcing actual language assembly from the user.
- **Oral Practice Sub-agent:** Leverages streaming voice patterns (configured via ADK's `Runner.run_live()` over WebSockets) to deliver audio evaluations and receive voice inputs.

3. Review & Critique Workflow (Loop Design Pattern)

This is a target **Deterministic Loop Workflow**. When a student provides an answer to an open-ended question inside the text/oral practice tracking blocks:

1. **Generator step:** The child node parses user output.
2. **Critique step:** A target analytical judge evaluates it against the strict educational facts indexed in **Vertex AI Search** and the mathematical laws recorded in the structural book schema.
3. **Termination Gate:** If the response shows clear misconceptions or leaves out crucial domain keywords, the framework cycles it back to the practice node with precise hints, continuing until an evaluation metric threshold (e.g., score ≥ 0.8) is attained or maximum iteration limits are hit.

4. Quiz Agent (Parallel Engine Pattern)

Generates custom assessments or timed exams by fanning out query generation requests across distinct book chapters concurrently via multi-threaded tool hooks, synthesizing them into structured test profiles.

5. Zatona Agent (Synthesis Expert)

An isolated agent optimized for summarizing large amounts of information. It uses extended context windows to deliver structured breakdowns, formula lookup guides, and comprehensive summaries directly to the learner.

Part 4: MongoDB Atlas MCP Implementation Specifics

To avoid complex manual database connector integrations within your agent source profiles, compile a declarative **Model Context Protocol Config** exposing your ingestion, analytics, exploration, and tracking pipelines as native execution actions.

Ingestion and Analysis Tool Definition

```

from mcp.server import Server
from pymongo import MongoClient

mcp_server = Server("fahem-mongodb-mcp")
client = MongoClient("mongodb+srv://...")
db = client.fahem_academic

@mcp_server.tool(name="ingest_extracted_metadata", description="Stores structured academic
data models returned by Gemini extraction workers.")
def ingest_extracted_metadata(dict)
    result = db.books.insert_one(book_profile_json)
    return f"Success: Academic entity verified and locked with ID: {result.inserted_id}"

```

Reporting and Aggregation Pipeline Tool Definition

To extract student performance analytics across distinct grade tiers without parsing documents line-by-line via the LLM, write a tool that offloads processing to MongoDB's highly optimized aggregation framework:

```

@mcp_server.tool(name="generate_student_insight_report", description="Aggregates student
performance metrics grouped by grade and subject parameters.")
def generate_student_insight_report(str, str)
    pipeline = [
        {"$match": {"grade": grade_tier, "subject_id": subject_id}},
        {"$group": {
            "_id": "$topic_id",
            "average_score": {"$avg": "$session_score"},
            "total_attempts": {"$sum": 1},
            "common_misconceptions": {"$push": "$primary_error_tag"}
        }},
        {"$sort": {"average_score": 1}}, // Highlight weakest concepts first
        {"$limit": 5}
    ]

```

```
]
return list(db.student_sessions.aggregate(pipeline))
```

Academic Space Search and Data Exploration Tool Definition

Expose Atlas Vector Search directly to the agent runtime, allowing semantic lookups across book components using hybrid indexes:

```
@mcp_server.tool(name="explore_academic_library", description="Performs vector-based
semantic search across core textbook chapters and keywords.")
def explore_academic_library(list, str)
    search_query = [
        {
            "$vectorSearch": {
                "index": "vector_index_spec",
                "path": "embedding",
                "queryVector": query_vector,
                "numCandidates": 50,
                "limit": 5
            }
        },
        {"$match": {"subject_id": subject_filter}}
    ]
    return list(db.question_bank.aggregate(search_query))
```

Part 5: Comprehensive Memory Framework & User State Tracking

To build user states that survive application context resets and maintain onboarding metrics without configuration decay, implement a three-tiered memory layer modeled on advanced ADK paradigms.

+-----+

THREE-TIER MEMORY LAYER	
+-----+	
1. Session Memory (Working Context)	
- Maps live turn transcripts, active chat lines, transient state.	
+-----+	
2. Persistent Store (User Profiling)	
- Locked to disk via DB. Handles language settings, onboarding profiles.	
+-----+	
3. Memory Bank (Vertex AI Semantic Recall Archive)	
- Stores past transcripts, cross-turn media, structural user inputs.	
+-----+	

Onboarding Continuity Workflow

If a student exits during onboarding to change the interface language or view a landing card, your tools must never write directly to private instance properties. Instead, leverage standard environment transaction states:

```
# Safe programmatic context modification framework inside ADK tools
@tool
def update_onboarding_checkpoint                                str                                dict

    # Fetch existing session state map safely
    current_state = context.state.get("onboarding_data", {})

    # Append newly captured user details
    current_state[step_name] = gathered_payload
    current_state["last_active_checkpoint"] = step_name

    # Persist back to the tracking token via context layer execution parameters
    context.state["onboarding_data"] = current_state
    return f"State synchronized up to step: {step_name}. Safe to transition across views."
```

Onboarding Logic Guards:

- **Phone Verification Hook:** Store a high-level key verification value (phone_verified: true) inside your core MongoDB user_profiles schema. When an onboarding session fires up,

check this key first. If it returns true, completely skip the verification screen unless an explicit "Reset Onboarding Session" execution request is requested by the user.

Part 6: Systematic Multi-Agent Evaluation Protocol

To transition your framework from general observations to production-grade verification metrics, adopt an automated **Three-Tier Testing Pyramid** using `agents-cli eval run`.

```

      /\
     /\  Tier 3: Human Review / Expert Blind Tests
    /----\
   /\  Tier 2: Trajectory Integration Tests (ADK Eval Trajectory)
  /-----\
 /\  Tier 1: Component Checks (Correct Tool Routing, Valid JSON formats)
/-----\
```

Tier 1: Component Checks

Verify your tools route correctly under baseline constraints. For example, pass a corrupted input string to your text-practice tool to ensure it returns explicit schema errors rather than leaking low-level stack traces.

Tier 2: Trajectory & Integration Evaluations

Construct a fixed evaluation profile configuration matrix (`tests/eval/eval_config.json`) along with automated test criteria instances (`tests/eval/evalsets/basic.evalset.json`):

```
{
  "test_cases": [
    {
      "input": "Can you quiz me on Grade 10 Arabic Grammar from the main student book?",
      "expected_trajectory": [
        "explore_academic_library",
        "fetch_book_chapters",

```

```

    "MCQ_Subagent_Invocation"
  ],
  "metrics_to_enforce": [
    "tool_trajectory_avg_score",
    "rubric_based_final_response_quality_v1",
    "hallucinations_v1"
  ],
  "thresholds": {
    "tool_trajectory_avg_score": 0.85,
    "rubric_based_final_response_quality_v1": 0.90
  }
}
]
}

```

Run `agents-cli eval run` continuously inside your **GitHub Action Deployment Workflow** (`deploy-agents.yml`). If any code modification alters target routing steps or introduces a hallucinated answer, the integration block flags the error, preventing the push to production.

Part 7: Architectural Optimization & UX Enhancements

Advanced Social Network & Academic Space Features

To transition the web framework from manual refresh routines to an interconnected learning environment, optimize your component tabs layout as follows:

1. Administration Panel Workspace

- **Ingestion Monitoring Node:** Provides detailed analytics on active Googlebot crawling instances, execution queues for Cloud Run workers, ingestion success metrics, and index generation sync tracking.
- **Content Policy Dashboard:** Monitors system-wide background operations via Eventarc, flagging user uploads that violate instructional guidelines or present non-academic material.
- **Token Allocation Control:** Directs token credit usage rates across tiered student tiers (e.g., Free, Basic, Premium Academic Space allowances).

2. Academic Environment Tab Navigation

- **The Connected Companion Interface:** Features a toggleable full-screen overlay mode that houses context alerts, task notification indicators, and active study tracking metrics.
- **Interactive Digital Library Module:** Enables split-view document inspection, horizontal swiping between core textbook pages, and full geometric visualization mapping.
- **Contextual Magnetization System:** Includes an interactive UI indicator that allows students to visually "magnetize" specific content blocks (such as a math problem from page 14 or an entire science chapter) to the parent Orchestrator Companion, pinning

that explicit node context to subsequent RAG lookups until manually cleared.

3. Enhanced Social Features Workspace

- **Real-time Collaboration Engine:** Upgrades core workspace views by incorporating active typing state triggers, instant notification alerts, and active discussion forums where student study groups can aggregate content and practice together using WebSockets or Firebase Realtime Database connections.
- **Onboarding Profile Management:** Features a refined avatar selection component supporting personalized vector assets or constrained identity image uploads (capped at 2MB via client-side size checks).

4. Interface Layout Realignment (Arabic RTL Support)

To fix navigation alignment breaks when toggling the system layout to Arabic, replace asymmetrical directional layouts with explicit CSS logical grouping parameters:

```
/* Update layout component alignments to prevent directional fracturing */
.landing-navbar-container {
  : flex;
    : center;
    : space-between;
  /* Replaces directional margins to support dual RTL/LTR rendering naturally */
  -inline-start: auto;
  -inline-end: 1.5rem;
  : row;
}

/* Ensure language toggles adapt typographic elements without layout overlap */
body[dir="rtl"] .landing-nav-links {
  : var(--font-arabic-calligraphy, sans-serif);
  : 0; /* Clear custom latin adjustments for clean Arabic display */
}
```

Part 8: Execution Blueprint & Testing Plan

To guarantee system stability before testing with heavy multi-volume textbook databases, implement this systematic step-by-step launch sequence:

- **Step 1: Localized Baseline Testing**
Configure a localized mock dataset consisting of one core document path (e.g., a

five-page excerpt from a Grade 10 math text). Verify your MongoDB Atlas Vector Search parameters return correct distance boundaries using manual tests.

- **Step 2: Execution Path Validation**

Deploy individual agent definitions to a secure development environment using `agents-cli deploy --prototype`. Run targeted validation scripts to ensure callback loops execute cleanly when text inputs or text-practice guardrails are explicitly engaged.

- **Step 3: Verification Check**

Execute an automated end-to-end regression evaluation scenario over the baseline math data. Once the validation checks confirm optimal tool selection rates and accurate citations (score ≥ 0.85), activate the complete ingestion system to safely compile the full textbook library.